SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

ANSWERS:

QUESTION 1: Online Banking System – Equivalence Partitioning, BVA and Random Testing

1. Problem Understanding and Analysis

The online banking system frequently fails during fund transfers, particularly at minimum and maximum transaction limits. This indicates that the existing test cases may not adequately cover boundary values, invalid inputs, and transaction-processing conditions.

Testing only normal transaction amounts is insufficient because defects often occur when input values are at or near permitted limits. The existing random testing approach may also miss these critical values.

2. Application of Black-Box Testing Techniques

A. Equivalence Partitioning (ECP)

ECP divides input data into valid and invalid equivalence classes. One representative value from each class is selected for testing.

Assume the permitted transfer amount is ₹100–₹50,000.

Test Case	Input Amount	Equivalence Class	Expected Result
TC1	₹50	Below minimum	Reject transfer
TC2	₹5,000	Valid range	Accept if other checks pass
TC3	₹60,000	Above maximum	Reject transfer
TC4	₹0	Invalid amount	Reject transfer
TC5	-₹100	Negative amount	Reject transfer

B. Boundary Value Analysis (BVA)

BVA tests values immediately below, at, and above important boundaries.

Test Case	Transfer Amount	Expected Result
TC1	₹99	Reject
TC2	₹100	Accept
TC3	₹101	Accept
TC4	₹49,999	Accept
TC5	₹50,000	Accept
TC6	₹50,001	Reject

These expected results assume that both stated limits are inclusive.

3. Evaluation of Random Testing

Random testing selects inputs without systematically targeting important boundaries.

Limitations:

- It may rarely select the exact minimum or maximum amount.
- It does not guarantee coverage of important input classes.
- It may miss rare transaction failures.

4. Improved Testing Strategy and Justification

The following strategy should be adopted:

1. Apply ECP to identify valid and invalid transaction inputs.
2. Apply BVA to test minimum and maximum transfer limits.
3. Test insufficient balance, invalid account numbers, and exceeded daily limits.
4. Simulate network interruptions and duplicate transfer requests.
5. Use automated regression tests after every defect correction.
6. Perform supplementary random testing to discover unexpected failures.

For every successful transfer, verify that the sender's balance decreases correctly, the recipient's balance increases correctly, and the transaction is recorded exactly once.

5. Conclusion

Combining ECP, BVA, transaction-integrity checks, and supplementary random testing provides systematic input coverage and improves defect detection. This approach reduces the risk of incorrect transfers, duplicate transactions, and financial losses

QUESTION 2: Healthcare Application – White-Box Testing and Coverage Criteria

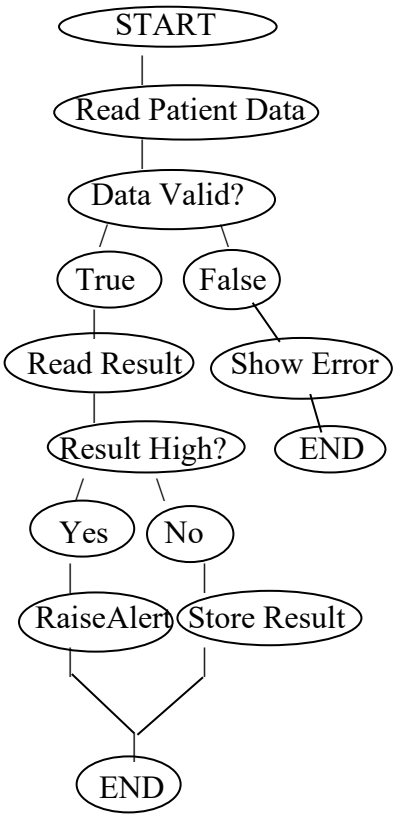
1. Problem Understanding and Analysis

The healthcare application developed critical defects after deployment because its testing process did not adequately cover the program's execution paths.

This suggests that the team may have tested only normal patient records without sufficiently testing abnormal results, invalid data, and alternative decision outcomes.

2. Construction of a Control Flow Graph (CFG)

Consider a module that validates patient data and checks laboratory results.



The graph contains decisions for patient-data validation and abnormal-result detection.

3. Coverage Analysis and Missing Paths

Statement coverage: Measures whether every executable statement has been executed at least once.

Branch coverage: Measures whether every decision outcome, including True and False, has been executed.

Path coverage: Measures whether the relevant execution paths through the program have been tested.

The following paths must be tested:

Test Case	Patient Data	Laboratory Result	Expected Outcome
TC1	Valid	High	Raise alert
TC2	Valid	Normal	Store result without abnormal-result alert
TC3	Invalid	Not processed	Display error and stop processing

For the simplified CFG, these three tests exercise the three feasible entry-to-exit paths.

Statement coverage alone is insufficient because executing every statement does not necessarily exercise both outcomes of every decision.

4. Improved Testing Strategy

1. Construct the CFG from the actual source code.
2. Identify all decisions and feasible execution paths.
3. Achieve 100% statement and branch coverage for the relevant module.
4. Use basis path testing to cover independent paths.
5. Test missing patient details, malformed results, and abnormal values.
6. Verify that alerts are generated correctly and invalid records cannot proceed.
7. Perform regression testing after fixing defects.

Coverage targets must be supplemented with requirement-based tests because even 100% branch coverage does not guarantee correct clinical behavior.

5. Conclusion

A structured white-box testing strategy identifies untested decision outcomes and execution paths. Combining coverage analysis with healthcare-specific validation and alert testing improves reliability and reduces the risk of incorrect medical decisions.

QUESTION 3: Retail E-Commerce System – ECP, BVA and Decision Table Testing

1. Problem Understanding and Analysis

The retail e-commerce system calculates totals using price, quantity, and discount. Errors in specific input combinations suggest that the existing testing approach does not adequately examine interactions between these fields.

Testing each input independently cannot guarantee that the combined calculation is correct. For example, a valid price and a valid discount may still produce an incorrect total when combined with an invalid quantity.

2. Equivalence Class Partitioning

Assume the business rules are:

- Price must be greater than ₹0.
- Quantity must be a positive integer.
- Discount must be between 0% and 50%, inclusive.

Input	Valid Equivalence Class	Invalid Equivalence Class
Price	Greater than ₹0	Zero or negative
Quantity	Positive integer	Zero, negative, or non-integer
Discount	0%–50%	Below 0% or above 50%

One representative input from each class is selected to reduce unnecessary testing.

3. Boundary Value Analysis

Input	Test Values	Expected Result
Price	₹0, ₹1, ₹2	Reject ₹0; accept positive values
Quantity	0, 1, 2	Reject 0; accept positive integers
Discount	-1%, 0%, 1%	Reject -1%; accept 0% and 1%
Discount	49%, 50%, 51%	Accept 49% and 50%; reject 51%

4. Decision Table Testing

Decision table testing evaluates combinations of conditions and their corresponding actions.

Condition	TC1	TC2	TC3	TC4
Price valid	Yes	Yes	No	Yes
Quantity valid	Yes	Yes	Yes	Yes
Discount valid	Yes	No	Yes	Yes

Expected outcome Calculate total Reject input Reject input Calculate total

The table covers representative rules. Additional rules should be included for stock availability, coupon eligibility, and discount restrictions if these affect the application.

The expected total is:

$$\text{Total} = (\text{Price} \times \text{Quantity})\left(1 - \frac{\text{Discount}}{100}\right)$$

For example, a product costing ₹1,000 with quantity 2 and a 10% discount should produce:

$$\text{Total} = 1000 \times 2 \times 0.9$$

$$\boxed{\text{Total} = ₹1{,}800}$$

5. Improved Testing Strategy and Justification

1. Apply ECP to classify valid and invalid inputs.
2. Apply BVA to test numeric limits.
3. Use decision tables to test important combinations.
4. Verify calculated totals against independently calculated expected values.
5. Test stock restrictions, discount eligibility, and rounding rules.
6. Automate the test cases for regression testing.

6. Conclusion

Combining ECP, BVA, and decision table testing provides systematic coverage of individual fields and their interactions. This approach reduces calculation errors and improves the correctness of e-commerce transactions.

QUESTION 4: Complex Software Module – Control Flow Graph and Path Testing

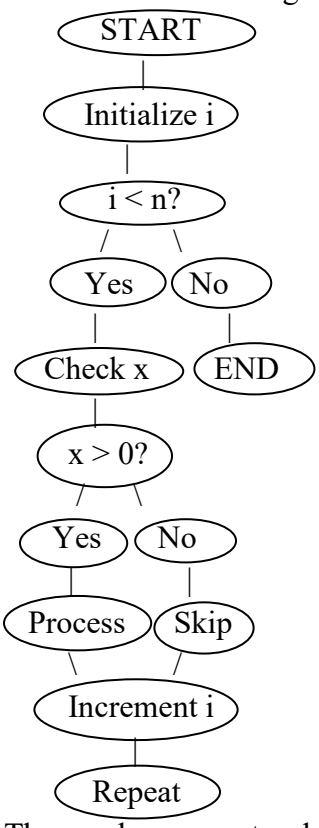
1. Problem Understanding and Analysis

The software module contains nested conditions and loops, but the testing team has achieved only partial path coverage.

This indicates that some independent execution paths, condition outcomes, or loop-exit scenarios have not been tested. Complex control structures require a systematic strategy rather than testing only typical execution paths.

2. Control Flow Graph

Consider the following logic:



The graph represents a loop containing a conditional statement.

3. Cyclomatic Complexity and Independent Paths

Cyclomatic complexity estimates the number of linearly independent paths.

$$V(G) = E - N + 2P$$

Where:

- E= Number of edges.
- N= Number of nodes.
- P= Number of connected components.

For a single connected graph:

$$V(G) = E - N + 2$$

For a connected graph with two binary decision nodes, assuming no other decisions:

$$V(G) = 2 + 1 = 3$$

Therefore, three linearly independent paths are required for a basis path set for this simplified graph.

4. Proposed Path Testing Strategy

Test Case	Input Condition	Path Tested	Expected Result
TC1	$n = 0$	Loop skipped	Program terminates
TC2	$n = 1, x > 0$	Loop entered; True branch	Process item
TC3	$n = 1, x \leq 0$	Loop entered; False branch	Skip item

These cases illustrate the main independent paths. Additional tests are needed for repeated iterations, nested decisions, and realistic input sequences.

5. Improved Strategy and Justification

- Construct a complete CFG from the source code.
- Identify all independent paths.
- Calculate cyclomatic complexity.
- Test both outcomes of every decision.
- Test loops with zero, one, two, and multiple iterations.
- Verify termination and loop-counter updates.
- Exclude infeasible paths from coverage claims with documented justification.

Complete path coverage may be impossible when loops allow an unbounded number of iterations. Basis path testing provides a practical alternative by targeting independent paths.

6. Conclusion

CFG-based basis path testing provides systematic control-flow coverage. Combined with loop-boundary and termination testing, it detects defects that partial path coverage may miss and improves the reliability of complex software modules.

QUESTION 5: Financial Application – Hybrid Black-Box and White-Box Testing

1. Problem Understanding and Analysis

The financial application uses both black-box and white-box testing, but defects continue to appear in production. This suggests that the techniques may not have been applied comprehensively or that the team has relied on coverage percentages without checking expected financial outcomes.

Possible gaps include untested transaction boundaries, missing execution paths, duplicate requests, and inadequate regression testing.

2. Black-Box Testing Strategy

Equivalence Partitioning: Divide transaction amounts into valid and invalid classes.

Boundary Value Analysis: Test values around the minimum and maximum transaction limits.

Assume the permitted amount is ₹100–₹50,000.

Test Case	Amount	Expected Result
TC1	₹99	Reject
TC2	₹100	Accept if other validations pass
TC3	₹101	Accept if other validations pass
TC4	₹49,999	Accept if other validations pass
TC5	₹50,000	Accept if other validations pass
TC6	₹50,001	Reject

3. White-Box Testing Strategy

Construct a CFG for the following simplified transaction logic:

Test all feasible paths:

1. Invalid account → reject.
2. Valid account with insufficient balance → reject.
3. Valid account with sufficient balance → transfer.

Use branch coverage to verify both outcomes of each decision and basis path testing to cover the independent paths.

4. Improved Hybrid Testing Strategy

1. Review the financial requirements and transaction rules.
2. Apply ECP and BVA to all important inputs.
3. Construct CFGs for transaction validation and processing.
4. Perform branch and basis path testing.
5. Test duplicate requests, network interruptions, and concurrent transactions.
6. Verify that failed transactions do not incorrectly alter balances.
7. Measure coverage and perform regression testing after every correction.

5. Analytical Justification

Black-box testing evaluates the externally observable behavior of the financial system. White-box testing examines the internal decision logic. Combining them reduces the chance that either input-related defects or control-flow defects will remain undetected.

Coverage metrics should be interpreted alongside requirement coverage, transaction invariants, and expected financial results.

6. Conclusion

A structured hybrid testing strategy improves input coverage, internal logic coverage, and financial correctness. It reduces production defects and increases confidence in transaction integrity.

QUESTION 6: HR Payroll System – Boundary Value Analysis and Equivalence Partitioning

1. Problem Understanding and Analysis

The HR payroll system calculates employee bonuses based on years of service and performance ratings. Incorrect payouts at exactly 5, 10, and 15 years indicate possible errors in boundary conditions or bonus-slab definitions.

For example, a comparison operator such as < may have been used when the business rule requires <=. However, the correct fix must be determined from the official payroll policy.

2. Equivalence Class Partitioning

For illustration, assume the bonus slabs are:

- Below 5 years: 5%.
- 5 to below 10 years: 10%.
- 10 to below 15 years: 15%.
- 15 years and above: 20%.

Service Period	Equivalence Class	Bonus Rate
0–4 years	Class 1	5%
5–9 years	Class 2	10%
10–14 years	Class 3	15%
15 years and above	Class 4	20%

These are illustrative rates; actual expected payouts must follow the organization's policy.

3. Boundary Value Analysis

Test immediately below, at, and above each boundary.

Years of Service	Expected Bonus Rate
4	5%
5	10%
6	10%

Years of Service	Expected Bonus Rate
9	10%
10	15%
11	15%
14	15%
15	20%
16	20%

If service is calculated using exact joining and payroll dates, also test the dates immediately before, on, and after each anniversary.

4. Additional Test Cases

- Test each service boundary with different performance ratings.
- Test zero and negative service values.
- Test invalid performance ratings.
- Verify bonus amounts for different salaries.
- Check rounding and currency calculations.
- Compare the calculated payout with an independently calculated expected value.

5. Why BVA and ECP Are Appropriate

Exploratory testing can discover unexpected defects, but it does not systematically guarantee that critical boundaries are checked.

BVA specifically targets boundary-related errors, while ECP identifies representative inputs from valid and invalid groups. These techniques provide a repeatable and auditable test suite for financial calculations.

6. Conclusion

ECP and BVA provide systematic coverage of service categories and critical boundaries. Combining them with performance-rating tests and regression testing helps prevent incorrect payroll payouts.

QUESTION 7: Laboratory Temperature Control – CFG and MC/DC

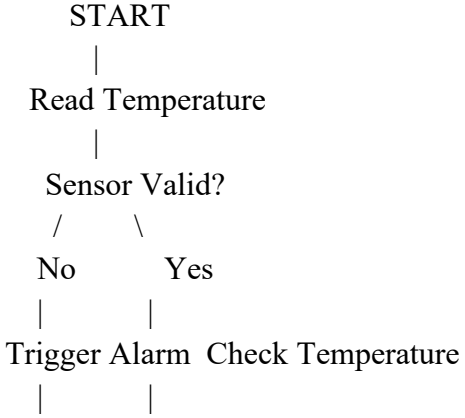
1. Problem Understanding and Analysis

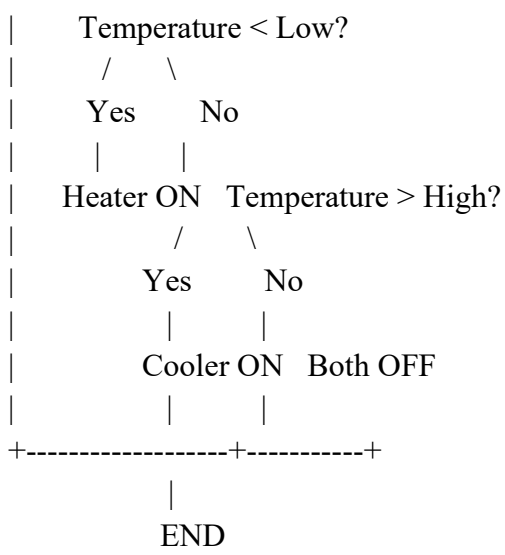
The laboratory temperature controller uses complex conditional logic to operate heaters, coolers, and alarms. An alarm failed to activate during a sensor-malfunction sequence, suggesting that certain branches or combinations of Boolean conditions were not tested.

Because this is a safety-critical system, testing must verify not only normal temperature control but also sensor failure and emergency behavior.

2. Control Flow Graph

Consider the following conceptual logic:





The exact graph should be refined using the actual program logic.

3. Branch Coverage Analysis

Branch coverage requires both outcomes of every decision to execute.

Test cases should include:

1. Invalid sensor → alarm activated.
2. Valid sensor and low temperature → heater activated.
3. Valid sensor and high temperature → cooler activated.
4. Valid sensor and normal temperature → heater and cooler remain off.

However, 100% branch coverage does not guarantee that every Boolean condition independently affects the decision.

4. MC/DC Testing

Consider the alarm decision:

$$A = S \vee T$$

Where:

- S= Sensor fault detected.
- T= Temperature exceeds a critical safety limit.
- A= Alarm activated.

Test	**Sensor Fault S**	**Critical Temperature T**	**Alarm A**
TC1	False	False	False
TC2	True	False	True
TC3	False	True	True

Independent effect of sensor fault: Compare TC1 and TC2. Only Schanges, and the alarm changes from False to True.

Independent effect of critical temperature: Compare TC1 and TC3. Only Tchanges, and the alarm changes from False to True.

Therefore, these tests demonstrate MC/DC for this two-condition OR decision.

5. Improved Testing Strategy

- Construct a complete CFG.
- Identify all Boolean conditions.
- Design independent condition pairs for every decision.

- Test sensor disconnection, invalid readings, and extreme temperatures.
- Verify alarms, safe shutdown, and actuator behavior.
- Perform hardware-software integration and regression testing.

6. Conclusion

MC/DC identifies logical defects that statement or branch coverage may miss. Combining MC/DC, CFG analysis, and hardware integration tests improves confidence in the safety and reliability of the temperature control system.

QUESTION 8: Travel Booking Portal – Decision Table Testing

1. Problem Understanding and Analysis

The travel booking portal allows multiple promotional codes, but some combinations produce a negative total amount. This indicates that the application may not validate interactions between booking price, promotion type, and user eligibility.

Testing each field independently cannot detect every combination-related defect.

2. Decision Table

Assume:

- The booking price must be positive.
- A promotion must be valid and applicable to the user's tier.
- Only one promotion can be applied unless stacking is explicitly permitted.
- The final amount cannot be negative.

Condition	TC1	TC2	TC3	TC4	TC5
Price valid	Yes	Yes	Yes	Yes	No
Promo valid	Yes	Yes	No	Yes	Yes
User eligible	Yes	No	Yes	Yes	Yes
Stacking allowed	Yes	No	No	No	No
Discount exceeds price	No	No	No	Yes	No
Expected result	Apply discount	Reject promo	Reject promo	Cap or reject according to policy	Reject booking

The table illustrates representative rules. Further rows or columns should be added if percentage and fixed discounts follow different eligibility rules.

3. Additional Test Scenarios

- Percentage discount with an eligible user.
- Fixed discount equal to the booking price.
- Fixed discount greater than the booking price.
- Two promotions applied together when stacking is prohibited.
- Expired promotional codes.
- Ineligible user tier.

4. Optimized Testing Strategy

1. Apply ECP to prices, promotional codes, and user tiers.
2. Apply BVA to booking prices and discount amounts.
3. Use decision tables to identify important condition combinations.
4. Use pairwise testing for additional combinations where exhaustive testing is too expensive, while retaining tests for critical business rules.

5. Verify the final amount against an independently calculated expected result.

A suitable calculation is:

$$\text{Final Amount} = \max(0, \text{Booking Price} - \text{Valid Discount})$$

This formula assumes that the business rules prohibit negative totals. It does not replace the need to reject invalid or unauthorized promotions.

5. Conclusion

Decision table testing systematically covers interactions among booking price, promotions, and user eligibility. Combined with boundary testing, it reduces redundant test cases while improving detection of invalid discounts and negative totals.

QUESTION 9: Database Migration Tool – Basis Path Testing and Loop Boundaries

1. Problem Understanding and Analysis

The legacy database migration tool reports high statement coverage but frequently crashes because of infinite loops.

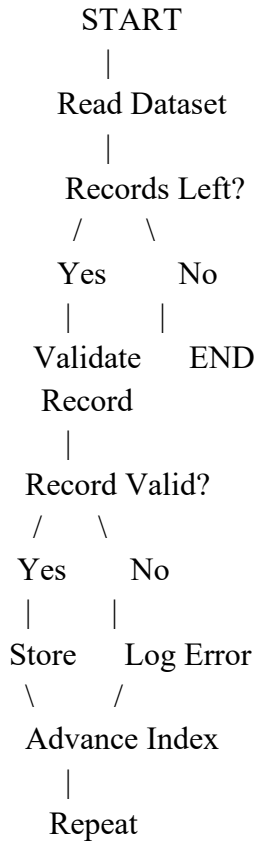
Statement coverage only confirms that statements execute. It does not prove that loops terminate correctly or that their exit conditions work for all input sizes.

2. Reasons for the Failure

- Loop-counter updates may be missing or incorrect.
- Exit conditions may never become true.
- Empty datasets may not be handled correctly.
- Nested loops may process malformed records incorrectly.
- Boundary values may cause the index to exceed the valid range.

3. Path Testing Strategy

Consider a loop that processes records until all records are validated.



Test the following cases:

Test Case	Dataset	Expected Result
TC1	Empty dataset	Terminate successfully
TC2	One valid record	Process one record and terminate
TC3	Multiple valid records	Process all records
TC4	Invalid record	Handle the error according to policy
TC5	Maximum supported dataset	Complete without hanging
TC6	Repeated or malformed records	Terminate safely

For every loop, test zero iterations where possible, one iteration, two iterations, and values near the maximum supported size.

4. Cyclomatic Complexity

Cyclomatic complexity estimates the number of linearly independent paths.

$$\boxed{V(G) = E - N + 2P}$$

For a single connected graph:

$$\boxed{V(G) = E - N + 2}$$

If a graph has three independent binary decisions and no additional decision nodes:

$$\boxed{V(G) = 3 + 1 = 4}$$

This value is illustrative; the actual complexity must be calculated from the real CFG.

5. Improved Testing Strategy

- Construct a CFG from the source code.
- Calculate cyclomatic complexity.
- Identify a basis set of independent paths.
- Test loop entry, continuation, and exit conditions.
- Add termination checks and timeouts.
- Use stress testing for large datasets.
- Perform regression testing after correcting loop logic.

6. Conclusion

Basis path testing combined with loop-boundary and termination testing detects defects that statement coverage alone may miss. It improves the reliability of database migration and reduces infinite-loop failures.

QUESTION 10: Food Delivery App – State Transition Testing

1. Problem Understanding and Analysis

The food delivery application moves through the states Ordered, Preparing, Out for Delivery, and Delivered. The application gets stuck when a driver cancels during delivery.

This indicates that the cancellation event and the corresponding state transition have not been handled correctly.

2. State Transition Model

Ordered
|
v
Preparing
|
v
Out for Delivery
|
v
Delivered

Out for Delivery
|
Driver Cancels
|
v
Reassign Driver
|
New Driver Assigned
|
v
Out for Delivery

The exact cancellation behavior must follow the business requirements. If reassignment is impossible, the system may instead cancel the order and initiate a refund.

3. State Transition Test Cases

Test Case	Current State	Event	Expected State
TC1	Ordered	Restaurant accepts	Preparing
TC2	Preparing	Order dispatched	Out for Delivery
TC3	Out for Delivery	Delivery completed	Delivered
TC4	Out for Delivery	Driver cancels	Reassign Driver
TC5	Reassign Driver	New driver assigned	Out for Delivery
TC6	Delivered	Driver cancellation	Remain Delivered; reject invalid transition
TC7	Out for Delivery	Reassignment fails	Apply defined cancellation or recovery policy

4. Improved Testing Strategy

- Identify all valid application states.
- Identify the events that cause state transitions.
- Test every valid transition.
- Test invalid transitions, including cancellation after delivery.
- Test driver reassignment failures and timeouts.
- Verify that notifications, refunds, and order records remain consistent.
- Perform regression testing after fixing the transition logic.

5. Why State Transition Testing Is Appropriate

ECP and BVA primarily focus on input values and boundaries. They may not reveal defects that occur only after a sequence of events.

State transition testing checks how the application behaves as it moves between states, making it especially suitable for dynamic workflows.

6. Conclusion

State transition testing identifies missing transitions and invalid state changes. It prevents orders from becoming stuck and improves the reliability of real-time delivery tracking.

QUESTION 11: Cybersecurity Authentication – Hybrid Testing and Error Guessing

1. Problem Understanding and Analysis

The authentication module uses password strength, OTP timing, and IP-location checks. Brute-force vulnerabilities discovered after testing suggest that the team did not adequately test repeated authentication failures, rate limiting, account lockout, or internal security decision paths.

Because authentication is security-critical, testing must examine both externally visible behavior and internal control logic.

2. Black-Box Testing

Test the authentication system using different input conditions.

Test Case	Input or Event	Expected Result
TC1	Correct password and valid OTP from permitted IP	Authenticate if all requirements pass
TC2	Incorrect password	Reject login
TC3	Expired OTP	Reject login
TC4	Reused OTP	Reject login
TC5	Restricted IP	Apply defined access-control policy
TC6	Repeated failed attempts	Apply rate limiting or lockout policy
TC7	Correct credentials after lockout	Follow account recovery policy

3. White-Box Path Analysis

The main feasible paths include:

1. Invalid password → reject.
2. Valid password, invalid OTP → reject.
3. Valid password and OTP, restricted IP → reject.
4. Valid password and OTP, permitted IP → authenticate.

Additional paths must cover rate limiting, lockout, recovery, and exceptional conditions.

4. Error Guessing and Improved Hybrid Strategy

Error guessing uses experience to anticipate likely failures.

Important scenarios include:

- Repeated login attempts.
- Reusing an OTP.
- Sending simultaneous login requests.
- Attempting authentication after account lockout.
- Bypassing an authentication step.
- Exploiting differences between failed-login responses.

The improved strategy should combine:

1. ECP for valid and invalid credentials.
2. BVA for OTP expiration times and retry limits.
3. Error guessing for likely security weaknesses.
4. CFG and basis path testing for authentication decisions.
5. Automated security testing and authorized penetration testing.
6. Regression tests to ensure that security fixes remain effective.

5. Conclusion

Hybrid testing combines external security checks with internal path analysis. Error guessing targets likely attack scenarios, while formal testing verifies that security controls operate correctly. This approach improves authentication reliability and reduces brute-force risk.

QUESTION 12: E-Learning Platform – ECP and Subscription Expiration

1. Problem Understanding and Analysis

The e-learning platform provides Free, Basic, and Premium access. Errors occur when users upgrade exactly on the expiration date.

This indicates that the subscription logic may not correctly distinguish between active, expiring, and expired subscriptions or may apply incorrect date comparisons.

2. Equivalence Class Partitioning for Dates

Assume the subscription policy states that access remains valid through the recorded expiration date.

Test Case	Date Condition	Expected Result
TC1	Before expiration	Existing access remains valid
TC2	On expiration date	Access follows the inclusive expiration policy
TC3	After expiration	Apply expired-subscription rules
TC4	Valid upgrade before expiration	Apply upgrade policy
TC5	Upgrade on expiration date	Apply the defined same-day upgrade policy
TC6	Upgrade after expiration	Apply renewal or reactivation policy

The actual expected result must be determined by the platform's subscription requirements.

3. Subscription-Type Equivalence Classes

Current Tier	Requested Tier	Expected Result
Free	Basic	Upgrade if eligible
Free	Premium	Upgrade if eligible
Basic	Premium	Apply upgrade policy
Premium	Basic	Apply downgrade policy
Any tier	Invalid tier	Reject request

4. Improved Test Design

- Apply ECP to subscription dates and membership types.
- Test the dates immediately before, on, and after expiration.
- Test upgrades, downgrades, renewals, and expired subscriptions.
- Check payment failures during upgrades.
- Test date-time boundaries, time zones, and leap years where relevant.
- Verify that course access matches the active subscription tier.

5. Justification

ECP identifies representative cases from valid and invalid input classes, reducing unnecessary repetition. However, because the failure occurs exactly on the expiration date, ECP should be supplemented with explicit date-boundary tests.

This combination addresses both subscription-category errors and date-comparison defects.

6. Conclusion

ECP combined with targeted date-boundary testing improves subscription-transition coverage. It helps prevent incorrect access restrictions and ensures consistent handling of upgrades and expirations.

QUESTION 13: Smart-Home Lighting Controller – Branch and Condition Coverage

1. Problem Understanding and Analysis

The lighting controller uses switch-case statements and complex Boolean expressions to adjust brightness based on ambient light readings.

Relying only on statement coverage is risky because all statements may execute without every decision outcome or Boolean condition being tested. Missing conditions can cause incorrect brightness settings or unsafe behavior when sensors fail.

2. Example Logic

```
IF sensor_valid AND ambient_light < threshold
   Increase brightness
ELSE IF sensor_valid AND ambient_light >= threshold
   Reduce brightness
ELSE
   Use safe default brightness
```

3. Branch Coverage

The following test cases cover the principal branches.

Test Case	Sensor Valid	Ambient Light	Expected Action
TC1	Yes	Below threshold	Increase brightness
TC2	Yes	At or above threshold	Reduce brightness
TC3	No	Any reading	Use safe default brightness

For switch-case statements, every relevant case and the default case must also be tested.

4. Condition Coverage

Condition coverage requires each Boolean condition to evaluate to both True and False.

Test Case	Sensor Valid	Light Below Threshold	Expected Action
TC1	True	True	Increase brightness
TC2	True	False	Reduce brightness
TC3	False	True	Safe default brightness
TC4	False	False	Safe default brightness

The example assumes that sensor validity is checked before any brightness adjustment.

5. Improved White-Box Testing Strategy

1. Construct the CFG from the actual code.
2. Measure statement and branch coverage.
3. Design tests to exercise each Boolean condition as True and False.
4. Test all switch-case branches and default handling.

5. Test sensor disconnection and extreme sensor readings.
6. Verify the output of the physical lighting hardware.
7. Perform integration and regression testing.

6. Conclusion

Branch and condition coverage expose missing logical outcomes that statement coverage may miss. Combining them with hardware-software integration testing reduces the risk of incorrect brightness control and sensor-related failures.

QUESTION 14: Logistics System – Robust Boundary Value Analysis

1. Problem Understanding and Analysis

The logistics system calculates shipping costs using weight, destination zone, and delivery speed. Errors occur when weights are zero or negative, showing that the current input-validation testing does not adequately cover invalid boundaries.

Incorrect handling may result in invalid shipping charges, failed bookings, or inconsistent records.

2. Robust Boundary Value Analysis

Assume the permitted weight range is greater than 0 kg and up to 100 kg.

Test Case	Weight	Expected Result
TC1	-0.1 kg	Reject
TC2	0 kg	Reject
TC3	0.1 kg	Accept
TC4	0.2 kg	Accept
TC5	99.9 kg	Accept
TC6	100 kg	Accept
TC7	100.1 kg	Reject

These expected results assume that the maximum weight of 100 kg is inclusive and that weights can be entered to one decimal place.

3. Additional Negative Test Cases

- Missing weight.
- Non-numeric weight.
- Invalid destination zone.
- Unsupported delivery speed.
- Weight within range combined with an invalid destination.
- Weight exceeding the maximum limit.

4. Improved Testing Strategy

1. Define the valid input ranges and business rules.
2. Apply ECP to classify valid and invalid values.
3. Apply robust BVA to test valid boundaries and out-of-range values.
4. Test combinations of weight, destination, and delivery speed.
5. Verify shipping charges using independently calculated expected values.
6. Check that invalid inputs produce clear error messages and do not create incorrect shipping records.
7. Perform regression testing after fixing validation defects.

5. Justification

Ordinary BVA tests values at and around valid boundaries. Robust BVA also includes invalid values outside the permitted range.

Negative testing is essential because real users may enter zero, negative, missing, or malformed values. The system must reject such inputs safely instead of performing incorrect calculations.

6. Conclusion

Robust BVA combined with ECP improves input validation and identifies boundary-related defects. It ensures that invalid weights are rejected and shipping costs are calculated only for valid inputs.

QUESTION 15: Medical Imaging Software – Basis Path Testing

1. Problem Understanding and Analysis

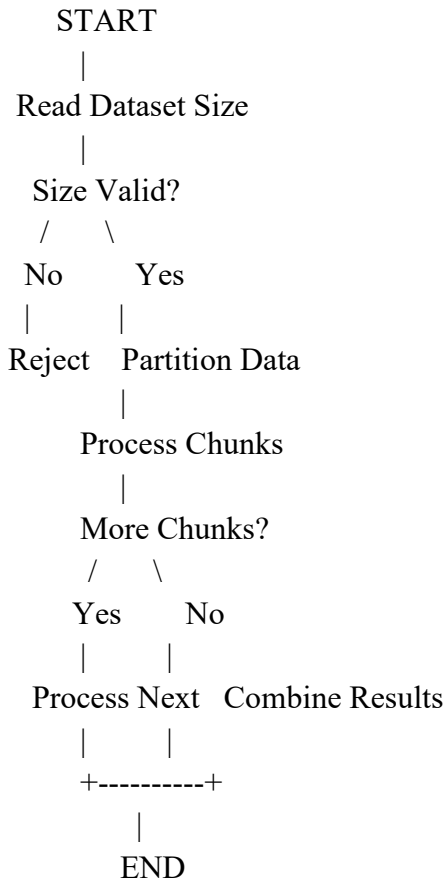
The medical imaging software processes large datasets using parallel loops. Inconsistent results occur only when the dataset size is a prime number.

This suggests a possible defect in loop indexing, workload partitioning, remainder handling, or parallel processing. Random testing may fail to detect this problem because it does not systematically select mathematically significant dataset sizes.

2. White-Box Testing and CFG

The testing team should examine the source code and construct a CFG representing the major decisions and loops.

A simplified conceptual flow is:



The actual CFG must reflect the implementation's parallel-processing logic.

3. Basis Path Testing

Basis path testing identifies linearly independent execution paths.

Cyclomatic complexity can be calculated using:

$$V(G) = E - N + 2P$$

For a single connected graph:

$$V(G) = E - N + 2$$

The calculated value provides the number of linearly independent paths in a basis set. It does not represent all possible paths when loops allow repeated execution.

4. Structured Test Cases

Test Case	Dataset Size	Purpose
TC1	0	Verify empty-input handling
TC2	1	Verify minimum valid dataset
TC3	2	Test a small prime-sized dataset
TC4	7	Test prime-sized partitioning
TC5	8	Compare with a non-prime size
TC6	100	Test a larger dataset
TC7	101	Test a larger prime-sized dataset

These sizes are illustrative and should be adjusted to the supported dataset range.

5. Improved Testing Strategy and Justification

1. Construct the CFG and identify independent paths.
2. Calculate cyclomatic complexity.
3. Perform basis path testing.
4. Test loop boundaries and array indexing.
5. Compare prime-sized and non-prime-sized datasets.
6. Compare parallel-processing outputs with a trusted sequential implementation.
7. Repeat tests with different thread counts.
8. Check race conditions, partitioning errors, numerical accuracy, and repeatability.
9. Perform regression testing after fixing defects.

Basis path testing verifies independent control-flow paths, while the prime-size test cases specifically investigate the observed mathematical pattern. Neither alone guarantees mathematical correctness, so numerical comparison and concurrency testing are also necessary.

6. Conclusion

Combining basis path testing, loop-boundary testing, numerical validation, and concurrency testing helps identify partitioning and indexing defects. This improves the accuracy and stability of medical imaging software and supports its safety-critical requirements.

Code of Conduct:

I **__Haritha D[192421282]__** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification